

Plans d'exécution et analyse



Samuel Rohaut

Objectifs

Comprendre ce qu'est un plan d'exécution

Apprendre à lire et interpréter les plans d'exécution

Identifier les opérations coûteuses

Reconnaître les opportunités d'optimisation

Plan d'exécution

Qu'est-ce qu'un plan d'exécution?

Un plan d'exécution est la feuille de route utilisée par le moteur de base de données pour exécuter une requête SQL.

Il détaille:

- L'ordre des opérations

- Les méthodes d'accès utilisées

- Les algorithmes de jointure choisis

- Les estimations de coût et de cardinalité

Qu'est-ce qu'un plan d'exécution?

Pourquoi l'analyser?

- Comprendre pourquoi une requête est lente

- Identifier les goulots d'étranglement

- Valider l'efficacité des index

- Vérifier si l'optimiseur fait les bons choix

Comment obtenir un plan d'exécution

PostgreSQL

-- Plan sans exécution

```
EXPLAIN SELECT * FROM title_basics WHERE start_year = 2020;
```

-- Plan avec exécution réelle

```
EXPLAIN ANALYZE SELECT * FROM title_basics WHERE start_year = 2020;
```

-- Options avancées

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
```

```
SELECT * FROM title_basics WHERE start_year = 2020;
```

Comment obtenir un plan d'exécution

MySQL / MariaDB

```
EXPLAIN SELECT * FROM title_basics WHERE start_year = 2020;
```

```
-- Format visuel (MySQL Workbench)
```

```
EXPLAIN FORMAT=TREE SELECT * FROM title_basics WHERE start_year = 2020;
```

Comment obtenir un plan d'exécution

SQL Server

```
-- Plan estimé
SET SHOWPLAN_ALL ON;
SELECT * FROM title_basics WHERE start_year = 2020;
SET SHOWPLAN_ALL OFF;

-- Plan avec statistiques d'exécution
SET STATISTICS PROFILE ON;
SELECT * FROM title_basics WHERE start_year = 2020;
SET STATISTICS PROFILE OFF;
```


Comment obtenir un plan d'exécution

Oracle

```
EXPLAIN PLAN FOR  
SELECT * FROM title_basics WHERE start_year = 2020;  
  
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Anatomie d'un plan d'exécution

```
Seq Scan on title_basics (cost=0.00..1356.00 rows=489 width=68)  
  Filter: (start_year = 2020)
```

Composants clés:

Type d'opération: Seq Scan (scan séquentiel)

Table cible: title_basics

Coût estimé: 0.00..1356.00 (démarrage..total)

Cardinalité estimée: 489 lignes

Largeur moyenne: 68 octets par ligne

Filtres appliqués: start_year = 2020

Anatomie d'un plan d'exécution

Comprendre les coûts

Unités arbitraires basées sur:

Coût de lecture séquentielle d'une page: `seq_page_cost`

Coût de lecture aléatoire d'une page: `random_page_cost`

Coût de traitement d'une ligne: `cpu_tuple_cost`

Coût de traitement d'un opérateur: `cpu_operator_cost`

Format du coût:

`startup_cost..total_cost`

Startup: coût avant la première ligne

Total: coût pour toutes les lignes

Anatomie d'un plan d'exécution : ANALYZE

```
Seq Scan on title_basics  (cost=0.00..1356.00 rows=489 width=68)
      (actual time=0.019..15.253 rows=524 loops=1)
    Filter: (start_year = 2020)
    Rows Removed by Filter: 8293812
```

Métriques additionnelles:

Temps réel: 0.019..15.253 ms (démarrage..total)

Lignes réelles: 524 (vs 489 estimées)

Nombre d'itérations: loops=1

Lignes filtrées: 8293812 éliminées par le filtre

Opérations fondamentales dans les plans

Méthodes d'accès aux tables

Sequential Scan (Seq Scan)

Lecture de toutes les pages de la table séquentiellement

Efficace pour les grands ensembles de résultats

Coûteux pour les recherches sélectives

```
Seq Scan on title_basics (cost=0.00..1356.00 rows=489 width=68)
```

Méthodes d'accès aux tables

Index Only Scan

Utilise uniquement l'index sans accéder à la table

Très efficace quand toutes les colonnes requises sont dans l'index

```
Index Only Scan using idx_title_basics_year_title on title_basics
      (cost=0.42..98.76 rows=489 width=24)
Index Cond: (start_year = 2020)
```

Méthodes de jointure

Nested Loop Join

Pour chaque ligne d'une table, parcourt l'autre table

Efficace pour petites tables ou avec index

```
Nested Loop (cost=0.86..352.82 rows=42 width=98)
-> Index Scan using title_ratings_pkey on title_ratings
    (cost=0.43..8.45 rows=1 width=20)
    Index Cond: (tconst = 'tt 0111161')
-> Index Scan using title_basics_pkey on title_basics
    (cost=0.43..344.36 rows=1 width=78)
    Index Cond: (tconst = title_ratings.tconst)
```


Méthodes de jointure

Hash Join

Crée table de hachage d'une table, parcourt l'autre

Efficace pour grandes tables et/ou sans index adaptés

```
Hash Join  (cost=958.86..18465.33 rows=42482 width=98)
  Hash Cond: (b.tconst = r.tconst)
    -> Seq Scan on title_basics b  (cost=0.00..14425.00 rows=525000
width=78)
    -> Hash  (cost=846.82..846.82 rows=8963 width=20)
      -> Seq Scan on title_ratings r
        (cost=0.00..846.82 rows=8963 width=20)
        Filter: (num votes > 10000)
```

Méthodes de jointure

Merge Join

Fusionne deux tables déjà triées

Efficace pour grandes tables déjà triées

```
Merge Join  (cost=9452.08..9578.60 rows=8963 width=98)
  Merge Cond: (b.tconst = r.tconst)
    ->  Sort  (cost=8546.51..8656.51 rows=44000 width=78)
          Sort Key: b.tconst
          ->  Seq Scan on title_basics b
                (cost=0.00..5425.00 rows=44000 width=78)
    ->  Sort  (cost=905.57..928.01 rows=8963 width=20)
          Sort Key: r.tconst
          ->  Seq Scan on title_ratings r
                (cost=0.00..546.63 rows=8963 width=20)
```

Opérations de traitement

Sort

Tri des résultats (ORDER BY, GROUP BY, jointures)

Potentiellement coûteux pour grands ensembles

```
Sort  (cost=10742.15..10847.11 rows=41982 width=36)
  Sort Key: r.average_rating DESC
-> Hash Join  (cost=958.86..8465.33 rows=41982 width=36)
    Hash Cond: (b.tconst = r.tconst)
```

Opérations de traitement

Aggregate

Calculs d'agrégation (GROUP BY, COUNT, SUM)

```
HashAggregate  (cost=746.84..756.84 rows=1000 width=12)
  Group Key: b.genres
-> Seq Scan on title_basics b  (cost=0.00..696.86 rows=19992 width=8)
```

Opérations de traitement

Limit

Restriction du nombre de lignes retournées

```
Limit    (cost=10847.11..10847.36 rows=100 width=36)
  ->    Sort    (cost=10742.15..10847.11 rows=41982 width=36)
          Sort Key: r.average_rating DESC
```

Analyse des divergences et problèmes

Mauvaise estimation de cardinalité

```
Limit  (cost=10847.11..10847.36 rows=100 width=36)
->  Sort  (cost=10742.15..10847.11 rows=41982 width=36)
      Sort Key: r.average_rating DESC
```

Estimation: 8963 lignes

Réalité: 89523 lignes

Problème: Statistiques obsolètes ou distribution atypique

Solution: Mettre à jour les statistiques (ANALYZE)

Choix d'accès inefficace

```
Seq Scan on title_ratings (cost=0.00..846.82 rows=50 width=20)  
  Filter: (average_rating > 9.0)
```

Problème: Scan séquentiel pour filtre très sélectif

Solution: Créer un index sur average_rating

Spill to disk (débordement sur disque)

```
Sort  (cost=1046987.57..1073237.14 rows=10499828 width=36)
      (actual time=86532.261..102458.935 rows=10499828 loops=1)
    Sort Key: b.primary_title
    Sort Method: external merge  Disk: 542336kB
```

Problème: Opération de tri trop volumineuse pour la mémoire

Solution: Augmenter work_mem ou retravailler la requête

Nested Loop inefficace

```
Nested Loop  (cost=0.86..4528521.82 rows=2345289 width=98)
              (actual time=0.091..235896.284 rows=2345289 loops=1)
```

Problème: Boucle imbriquée sur grande table sans index

Solution: Créer index ou forcer jointure par hachage

Points d'attention dans l'analyse

Opérations séquentielles vs indexées

Règle empirique: Seq Scan généralement optimal si >5-10% des lignes sont retournées

Attention aux scans séquentiels répétés sur grosses tables

Points d'attention dans l'analyse

Estimation vs réalité

Écarts importants indiquent statistiques obsolètes ou problème de modélisation

Surveiller rows (estimées) vs actual rows (réelles)

Points d'attention dans l'analyse

Opérations de tri et de hachage

Méthode: "quicksort in memory" vs "external merge Disk"

Débordements sur disque (spill to disk) = problèmes de performance

Points d'attention dans l'analyse

Temps d'exécution

Startup time: temps pour première ligne

Total time: temps pour résultat complet

Important pour applications interactives vs traitements batch

Optimisation basée sur les plans d'exécution

Workflow d'analyse

Identifier l'opération la plus coûteuse (temps ou ressources)

Vérifier si l'estimation correspond à la réalité

Évaluer les méthodes d'accès et de jointure

Identifier les opportunités d'indexation

Tester les modifications et comparer les plans

Optimisation basée sur les plans d'exécution

Exemples d'optimisations courantes

Remplacer Seq Scan par Index Scan (créer index)

Remplacer Index Scan par Index Only Scan (index covering)

Remplacer Nested Loop par Hash Join (ou inverse selon contexte)

Éviter les tris externes (augmenter work_mem)

Corriger les estimations (ANALYZE, statistiques étendues)

Mise en pratique avec le TP #02